Agriculture Triagent Agent & Quiz System

A Project-II Report

Submitted in partial fulfilment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Aaditya Verma – EN22CS301009
Aakansha Rajani-EN22CS301012
Abhijeet Patidar-EN22CS301029
Aditya Pandey-EN22CS301057
Aditya Thakre-EN22CS301069

Under the Guidance of
Prof. Ajaj Khan
Prof. Nishant Shrivastava
Dr. Ritesh Joshi

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331 JAN-JUNE 2026

Agriculture Triagent Agent & Quiz System

A Project-II Report

Submitted in partial fulfilment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Aaditya Verma – EN22CS301009
Aakansha Rajani-EN22CS301012
Abhijeet Patidar-EN22CS301029
Aditya Pandey-EN22CS301057
Aditya Thakre-EN22CS301069

Under the Guidance of
Prof. Ajaj Khan
Prof. Nishant Shrivastava
Dr. Ritesh Joshi

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331 JAN-JUNE 2026

Report Approval

The project work Agriculture Triagent Agent & Quiz System is hereby approved as a creditable study of an engineering subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approved any statement made, opinion expressed, or conclusion drawn there in; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner Name:

Designation:

Affiliation:

External Examiner Name:

Designation:

Affiliation:

Declaration

We hereby declare that the project entitled **Agriculture Triagent Agent & Quiz System** submitted in partial fulfilment for the award of the degree of Bachelor of Technology in Computer Science and Engineering completed under the supervision of **Prof. Ajaj Khan, Prof. Nishant Shrivastava, Dr. Ritesh Joshi** Faculty of Engineering, Medicaps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of student(s) with date

Aaditya Verma – EN22CS301009 _____

Aakansha Rajani- EN22CS301012 _____

Abhijeet Patidar – EN22CS301029 _____

Aditya Pandey- EN22CS301057 _____

Aditya Thakre – EN22CS301069 _____

Certificate

We, **Prof. Ajaj Khan, Prof. Nishant Shrivastava, Dr. Ritesh Joshi** certify that the project entitled **Agriculture Triagent Agent & Quiz System** submitted in partial fulfilment for the award of the degree of Bachelor of Technology by **Aaditya Verma, Aakansha Rajani, Abhijeet Patidar, Aditya Pandey, Aditya Thakre** is the record carried out by them under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Ajaj Khan,	**Mr. Suraj Nayak**
Prof. Nishant Shrivastav,	**Mr. Ashwin Krishnan**
Dr. Ritesh Joshi	**Mr Vaibhav**
Faculty of Engineering	
Medicaps University, Indore	

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Acknowledgements

We would like to express my deepest gratitude to Honourable Chancellor, **Shri R C Mittal,** who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Dr. D. K. Patnaik,** Vice Chancellor, Medicaps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Dr. Ratnesh Litoriya,** Associate Dean of computing and Allied Branches, Medicaps University, for giving me a chance to work on this project. I would also like to thank my Head of Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Student may write as per their experience.

Aaditya Verma – EN22CS301009
Aakansha Rajani – EN22CS301012
Abhijeet Patidar-EN22CS301029
Aditya Pandey-EN22CS301057
Aditya Thakre-EN22CS301069

B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Abstract

Agriculture Triagent & Quiz System is an innovative final integrated project that combines the capabilities of **Generative AI, Agentic AI, and DevOps** to transform traditional manufacturing ideation, design, and deployment processes into an intelligent automated workflow. The system enables users to enter manufacturing product ideas in natural language and receive complete multimodal outputs such as product descriptions, functional specifications, supplier insights, structured reports, and AI-generated prototype images.

The project integrates multiple specialized AI agents including a **Researcher Agent** for sourcing manufacturing data, a **Writer Agent** for generating professional reports, and an orchestration module that coordinates workflow execution. It also leverages **Large Language Models (LLMs)** for prompt enhancement, text generation, and decision support, while image generation models create realistic visual prototypes of products.

To improve relevance and reduce hallucinations, the system incorporates a **Vector Database** for semantic search and contextual retrieval. Users can interact through a modern web-based dashboard where they can submit prompts, monitor real-time progress, view outputs, and download reports.

From the deployment perspective, the platform follows modern **DevOps practices** using Docker containerization and Kubernetes orchestration for scalability, high availability, and zero-downtime rolling updates. The stateless microservice architecture ensures efficient performance, fault tolerance, and easy future expansion.

This project demonstrates how AI-powered automation can revolutionize manufacturing workflows by reducing manual effort, accelerating decision-making, enhancing creativity, and enabling smart industrial transformation. The **Agriculture Triagent & Quiz System** serves as a complete intelligent ecosystem for next-generation manufacturing design, planning, and deployment.

Table of Contents

Chapter	Topic	Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi
	Table of Contents	vii
	List of Figures	viii
	List of Tables	ix
	Abbreviations	x
Chapter 1	Introduction	1
	1.1 Introduction	1
	1.2 Literature Review	1 – 2
	1.3 Objectives	2
	1.4 Significance	2 – 3
	1.5 Research Design	3
	1.6 Source of Data	3
	1.7 Chapter Scheme	3
Chapter 2	Requirements Specification	4
	2.1 User Characteristics	4
	2.2 Functional Requirements	4 – 5
	2.3 Dependencies	5
	2.4 Performance Requirements	5
	2.5 Hardware Requirements	5 – 6
	2.6 Constraints and Assumptions	6
Chapter 3	Design	7
	3.1 Algorithm	7
	3.2 Function Oriented Design for Procedural Approach	7
	3.3 System Design	7 – 8
	3.3.1 Data Flow Diagram	8 – 9
	3.3.2 Activity Diagram	10
	3.3.3 Class Diagram	11
	3.3.4 ER Diagram	12
	3.3.5 Use Case Diagram	13

Chapter	Topic	Page No.
Chapter 4	Implementation, Testing and Maintenance	14
	4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation	14
	4.2 Testing Techniques and Test Plans	14 – 16
	4.3 Installation Instructions	16
	4.4 End User Instructions	16
Chapter 5	Results and Discussions	17
	5.1 User Interface Representation	17
	5.2 Brief Description of Various Modules of the System	17
	5.3 Snapshots of System with Brief Detail of Each	18 – 19
Chapter 6	Summary and Conclusion	20
Chapter 7	Future Scope	21
	Appendix	22 – 24
	Reference	25

CHAPTER 1

INTRODUCTION

1.1 Introduction

AgriTriage is a unified artificial intelligence platform designed to address two critical challenges in modern agriculture: rapid emergency response to farmer distress messages, and continuous education of farming communities through AI-powered learning tools. The platform integrates two distinct but complementary intelligent subsystems — an Agentic Triage pipeline that processes natural language messages from farmers and generates structured, prioritized responses, and a GenAI-powered quiz system called Farm Wise that enables farmers and agricultural workers to test and expand their knowledge on crop management, pest control, irrigation, and allied topics.

Agriculture in India and globally is highly vulnerable to time-sensitive crises. A delayed response to a crop disease outbreak, a flooding event, or a pest infestation can result in catastrophic yield losses. Traditional support systems that rely on human agents reading messages and manually triaging requests are slow and prone to inconsistency. AgriTriage addresses this gap by deploying a four-stage large language model (LLM) pipeline that automatically classifies urgency, extracts named entities from the message (such as farmer ID, crop type, and location), generates an empathetic draft response, and produces a one-line internal summary — all in a single API call, typically in under two seconds.

The platform is built on FastAPI for its backend, uses LangChain for orchestrating the multi-step agentic pipeline, and is powered by Groq's inference infrastructure running Meta's Llama 3 family of models. The frontend is a dark-themed, single-page web application written in vanilla HTML, CSS, and JavaScript, accessible directly from a browser without requiring any installation by the end user.

1.2 Literature Review

The development of AgriTriage draws on several intersecting bodies of research and technology. The concept of agentic AI, where complex tasks are decomposed into sequential subtasks handled by specialized large language model interactions, is supported by modern prompt engineering and task orchestration frameworks provided by LangChain [2][3]. The practical implementation of such agentic workflows is further enabled through integration frameworks that support modular execution of AI pipelines. These concepts form the theoretical and technical foundation behind the multi-stage triage pipeline developed in this project.

Named Entity Recognition (NER) applied to agricultural contexts has been studied as part of agricultural information extraction systems. Traditional NER approaches relied heavily on manually annotated datasets and domain-specific training. However, the emergence of instruction-tuned large language models such as Llama has made zero-shot entity extraction possible in real-world applications. By leveraging Meta’s Llama models through Groq’s high-

speed inference infrastructure, the system is capable of extracting entities such as farmer identification numbers, crop types, and geographic locations directly from unstructured text inputs [1][8].

The application of artificial intelligence in agricultural advisory systems has shown significant potential in improving communication between farmers and support agencies. Existing advisory systems have primarily focused on reactive support mechanisms, often relying on human intervention. AgriTriage extends this concept by integrating generative AI capabilities for automated urgency classification, structured information extraction, and response generation. The backend implementation of these services is supported using FastAPI and Python-based APIs for scalable deployment [6][9].

On the educational side, the FarmWise module incorporates AI-driven learning experiences through quiz-based interaction and dynamic knowledge generation. The web-based delivery of these educational tools is supported using modern frontend technologies such as HTML, CSS, and JavaScript, while learner performance and session persistence are managed using Supabase cloud infrastructure. This architecture supports interactive learning and long-term knowledge retention among farming communities [5][10].

1.3 Objectives

The primary objective of this project is to design and implement an integrated AI platform that serves agriculture support operations with both immediate response automation and long-term knowledge building. More specifically, the project aims to develop an automated triage pipeline that processes farmer messages and produces structured, urgency-ranked responses without human intervention. It also aims to build a generative AI quiz system that dynamically explains answers and provides personalized feedback to learners. A further objective is to integrate both systems into a unified web interface that is simple enough for non-technical agricultural extension workers to operate. Finally, the project aims to persist quiz session data in a cloud database and expose a public leaderboard to encourage competitive learning.

1.4 Significance

The significance of this project lies in its potential to materially reduce the response latency in agricultural emergency situations. By automatically classifying incoming messages as HIGH, MEDIUM, or LOW urgency and generating a professional draft response, the system allows human operators to focus their attention on genuinely complex cases rather than spending time triaging and composing routine replies. This is particularly valuable in contexts where agricultural helpline operators manage hundreds of messages per day.

On the educational side, the FarmWise quiz module democratizes access to expert agricultural knowledge by wrapping it in an engaging, interactive format. The AI-generated explanations for each quiz question go beyond simple right/wrong feedback, providing contextual knowledge that helps farmers understand not just the correct answer but the reasoning behind it — an approach that has been shown to improve knowledge transfer.

1.5 Research Design

This project follows a design-and-build research methodology. The research began with an analysis of existing agricultural support systems and their limitations, leading to the identification of key functional requirements. Prototypes were built iteratively: the triage pipeline was implemented first, tested with representative farmer messages covering diverse scenarios (flood emergencies, pest reports, crop diseases, loan queries), and refined based on output quality. The FarmWise quiz module was developed in parallel, with the leaderboard feature added after the core quiz workflow was validated. The frontend was developed last, integrating both modules under a single tabbed interface.

1.6 Source of Data

The system does not rely on a static domain-specific dataset for inference. Instead, it leverages the pre-trained knowledge of large language models (Llama 3.1-8b-instant for the triage agent and Llama 3.3-70b-versatile for the quiz and Q&A functions) via the Groq API. The quiz question pool is maintained as a structured Python list within the genai_routes.py file, which can be extended by administrators. Leaderboard and quiz session data have persisted in a Supabase PostgreSQL database. Environment variables stored in a .env file manage API credentials for Groq and Supabase.

1.7 Chapter Scheme

The remainder of this report is organized as follows.

Chapter 2 presents the requirements specification including user characteristics, functional and non-functional requirements, hardware and software dependencies, and system constraints.

Chapter 3 covers the system design, including the data flow diagrams, activity diagram, class diagram, ER diagram, use case diagram, and database design.

Chapter 4 addresses implementation, covering the technology stack, testing strategy, installation instructions, and end-user guide.

Chapter 5 presents the results and discussions, including screenshots of the system and descriptions of each module.

Chapter 6 provides a summary and conclusions, and Chapter 7 outlines the scope for future enhancement.

CHAPTER 2

REQUIREMENTS SPECIFICATION

2.1 User Characteristics

The AgriTriage platform is designed for multiple categories of users involved in agricultural support, education, and system management. The primary users of the system are agricultural support officers or helpline operators who are responsible for handling farmer queries and emergency reports. These users typically possess basic to moderate digital literacy and are familiar with using web applications, communication platforms, and data entry systems. Their main interaction with the platform involves submitting farmer messages, reviewing AI-generated urgency classifications, analyzing extracted entities, and sending appropriate responses based on the system recommendations.

The second category of users includes farmers, agricultural field workers, and learners who directly interact with the educational features of the platform. These users may not possess advanced technical knowledge but are expected to have basic smartphone or web browsing skills. They use the FarmWise module to participate in agricultural quizzes, ask farming-related questions, and gain knowledge related to crop management, pest control, irrigation, fertilizers, and sustainable farming practices.

The third category of users includes system administrators and developers responsible for maintaining the technical infrastructure of the platform. These users require knowledge of Python, FastAPI, API integration, database management, and cloud-based services such as Supabase. Their responsibilities include monitoring system performance, managing quiz content, maintaining API credentials, and updating application modules when required.

2.2 Functional Requirements

The AgriTriage system is expected to perform several functional operations to support intelligent agricultural assistance. The first major functionality of the system is the triage process. The system must accept a farmer's distress message through the frontend interface and process it using the agentic AI pipeline. After processing, the system should classify the urgency level of the message, assign an urgency score, identify the intent of the query, extract important entities such as crop names, locations, dates, and farmer information, and generate a professional response draft.

The second functionality of the system is the FarmWise educational module. This module allows users to select quiz topics and difficulty levels, attempt multiple-choice questions, receive AI-generated explanations, and view performance feedback. The system should also store quiz session details in the cloud database and display leaderboard rankings for encouraging competitive learning.

2.3 Dependencies

The AgriTriage platform depends on a combination of software frameworks, cloud services, APIs, and development tools. The backend of the system is developed using FastAPI, which provides high-performance API routing and request handling. Uvicorn is used as the ASGI server for executing the FastAPI application efficiently. Python serves as the primary programming language for implementing business logic, AI integration, and backend services.

The agentic AI workflow is implemented using LangChain, which manages prompt orchestration, task decomposition, and communication between multiple reasoning stages. The language processing capabilities are powered by Groq's inference platform using Meta's Llama language models for fast and accurate response generation.

Supabase is used as the cloud database solution for storing quiz session records, leaderboard information, and other persistent application data. The frontend of the system is developed using HTML, CSS, and JavaScript, which provide a responsive and interactive user experience without requiring additional frontend frameworks.

Version control and project collaboration are managed using Git and GitHub. Environment configuration files are used to securely store API keys, database credentials, and runtime configuration settings.

2.4 Performance Requirements

The AgriTriage system is designed to provide efficient and near real-time responses under normal operating conditions. The triage module is expected to process user messages quickly and return structured outputs without noticeable delays. The use of Groq's low-latency inference infrastructure ensures faster execution of multiple language model calls required in the agentic pipeline.

The FarmWise quiz module is designed to generate quiz questions, evaluate responses, and display results within a short response time. Database operations such as saving quiz sessions and retrieving leaderboard information are optimized using cloud-based storage services.

The system is designed to support multiple concurrent users without affecting response quality. Since the application follows a stateless API architecture, it can be scaled horizontally when the number of users increases.

2.5 Hardware Requirements

The development and deployment of the AgriTriage platform require a computing system with a modern processor, a minimum of 4 GB RAM, and stable internet connectivity. The server hosting the backend should support Python execution and cloud API communication.

For client-side usage, users require a device such as a desktop computer, laptop, tablet, or smartphone with an updated web browser including Google Chrome, Mozilla Firefox,

Microsoft Edge, or Safari. Since AI processing is performed using cloud-based services, no specialized local GPU hardware is required.

A reliable internet connection is necessary for communication between the frontend application, backend services, Groq inference APIs, and Supabase cloud database services.

2.6 Constraints and Assumptions

The AgriTriage system operates under certain technical and operational constraints. The platform is dependent on external cloud services such as Groq and Supabase. Any service outage, API quota limitation, or network connectivity issue may temporarily affect system functionality.

The current version of the system is optimized primarily for English-language inputs, as the prompt engineering and response formatting are designed around English communication. Support for regional languages can be added in future versions through multilingual prompt optimization.

The quiz question pool is currently maintained by the administrator and updated through backend configuration files. Dynamic quiz content management through an admin dashboard is not included in the current implementation.

It is assumed that users provide meaningful and valid input messages to ensure accurate AI-generated responses. The system also assumes that API credentials, environment variables, and database configurations are correctly set before deployment.

CHAPTER 3

DESIGN

3.1 Algorithm

The system operates using a hybrid algorithm that integrates Generative AI processing with an agent-based execution model. When a user submits a query through the frontend interface, the request is first received by the backend server built using FastAPI. The system validates the input to ensure that it is not empty and follows the expected format. After validation, the request is passed to the processing layer, where the system determines the nature of the query.

If the query is straightforward and requires a direct response, it is forwarded to the generative AI module, which communicates with the external Large Language Model API. The API processes the input and returns a response that is then formatted and sent back to the user.

In cases where the query is complex and requires multiple steps or logical reasoning, the agentic module is activated. This module decomposes the problem into smaller sub-tasks. Each sub-task is processed either sequentially or in parallel depending on dependencies. The intermediate outputs are combined to form the final response. The system ensures that the response is coherent and relevant before delivering it to the frontend.

This algorithm ensures efficiency by dynamically selecting the processing approach based on the complexity of the user query while maintaining scalability and modularity.

3.2 Function Oriented Design for Procedural Approach

The system is structured using a function-oriented design where each component is divided into smaller functional modules. The input handling function is responsible for capturing user queries from the frontend interface and sending them to the backend. This function ensures that user input is properly formatted and transmitted securely.

The validation function checks the correctness of the input and ensures that no invalid or harmful data is processed. Once validated, the routing function determines whether the request should be handled by the generative AI module or the agentic module.

The generative processing function communicates with the Large Language Model API to generate responses for simple queries. The agentic processing function, on the other hand, handles complex queries by breaking them down into smaller tasks, executing each task, and combining the results.

The response formatting function ensures that the output is structured and readable before sending it back to the frontend. Each function operates independently, allowing the system to be easily maintained and extended.

3.3 System Design

The system follows a layered architecture that separates the user interface, application logic, and AI processing components. The frontend layer is responsible for user interaction and is

implemented using HTML, CSS, and JavaScript. It provides an intuitive interface where users can input queries and view responses.

The backend layer acts as the core processing unit and is implemented using FastAPI. It handles request routing, validation, and communication with external AI services. This layer ensures efficient handling of multiple user requests.

The AI processing layer consists of two main components: the generative AI module and the agentic AI module. The generative module is responsible for producing natural language responses, while the agentic module manages task decomposition and execution. The interaction between these layers ensures that the system can handle a wide range of queries effectively.

3.3.1 Data Flow Diagram

Level 0 Data Flow Diagram represents the system as a single process. The user provides input through the interface, which is processed by the system and sent to the AI module. The processed output is then returned to the user. This diagram highlights the overall flow of data without going into internal details.

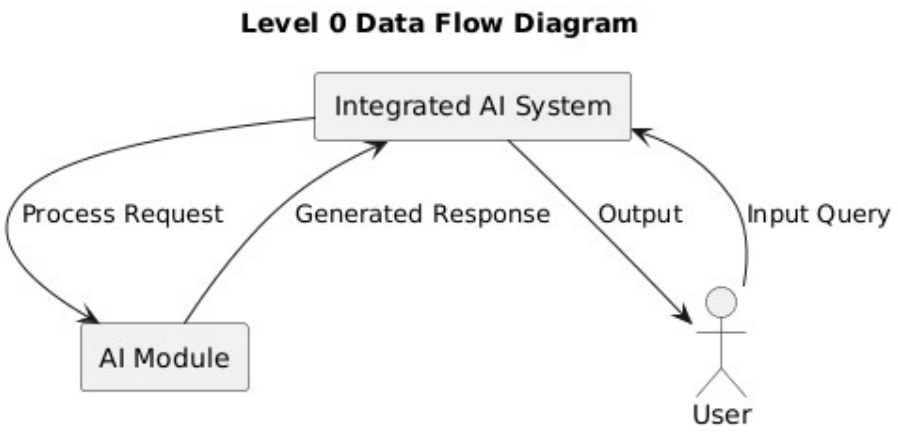

Fig No -3.1
Level 0 Data Flow Diagram

Level 1 Data Flow Diagram provides a more detailed view of the system. The user input is first handled by the input module and then passed to the validation module. After validation, the routing module decides whether the request should be processed by the generative AI module or the agentic module. The generative module communicates with the LLM API, while the agentic module performs task decomposition and execution. The results are then combined and passed to the output module, which sends the final response to the user.

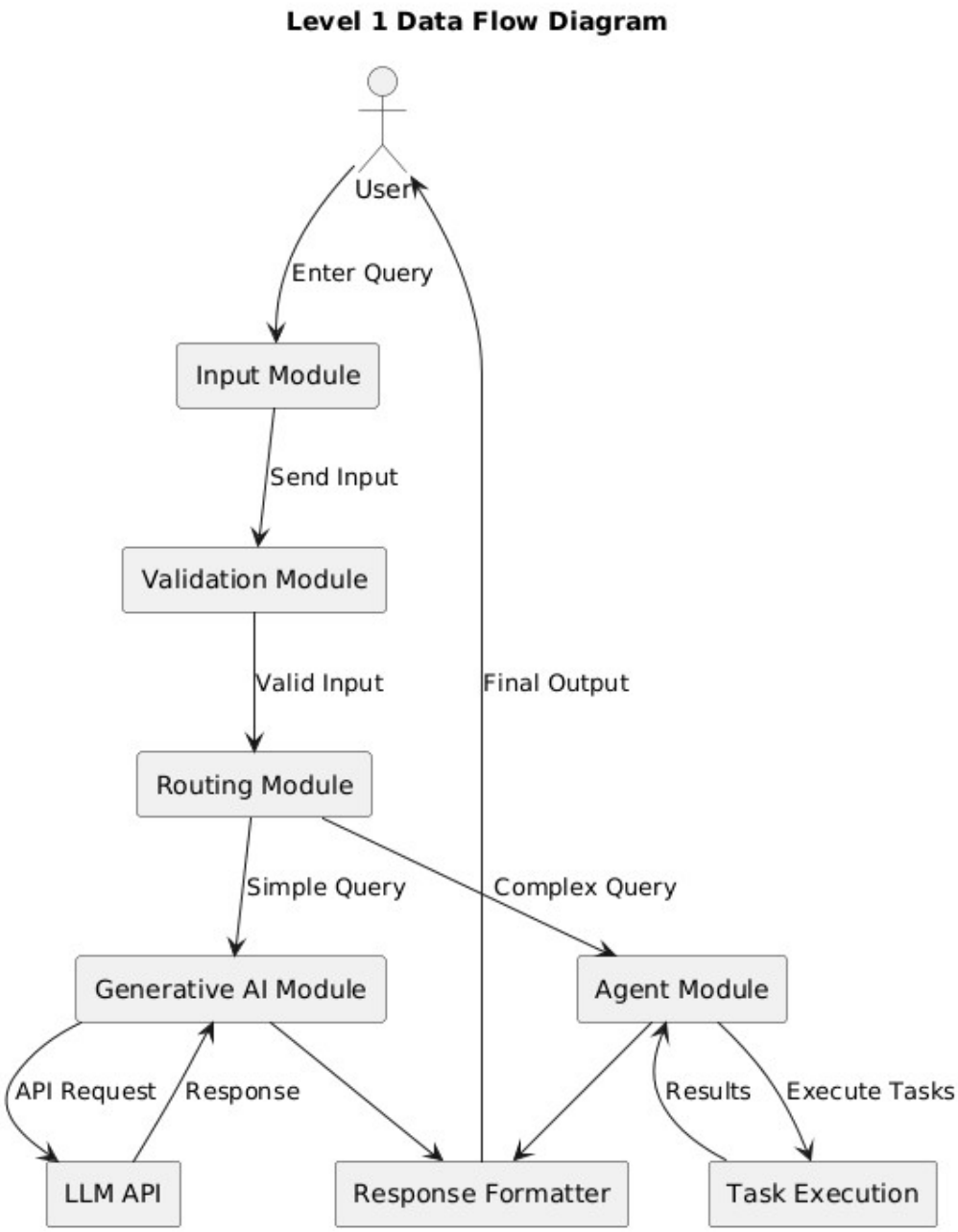

Fig No -3.2
Level 1 Data Flow Diagram

3.3.2 Activity Diagram

The activity flow begins when the user enters a query into the system. The system receives the input and validates it. If the input is invalid, the system prompts the user to re-enter the query. If the input is valid, the system determines whether the query is simple or complex.

For simple queries, the system directly sends the request to the generative AI module and retrieves the response. For complex queries, the system activates the agentic module, which breaks the task into smaller steps, executes them, and aggregates the results. The final response is then displayed to the user.

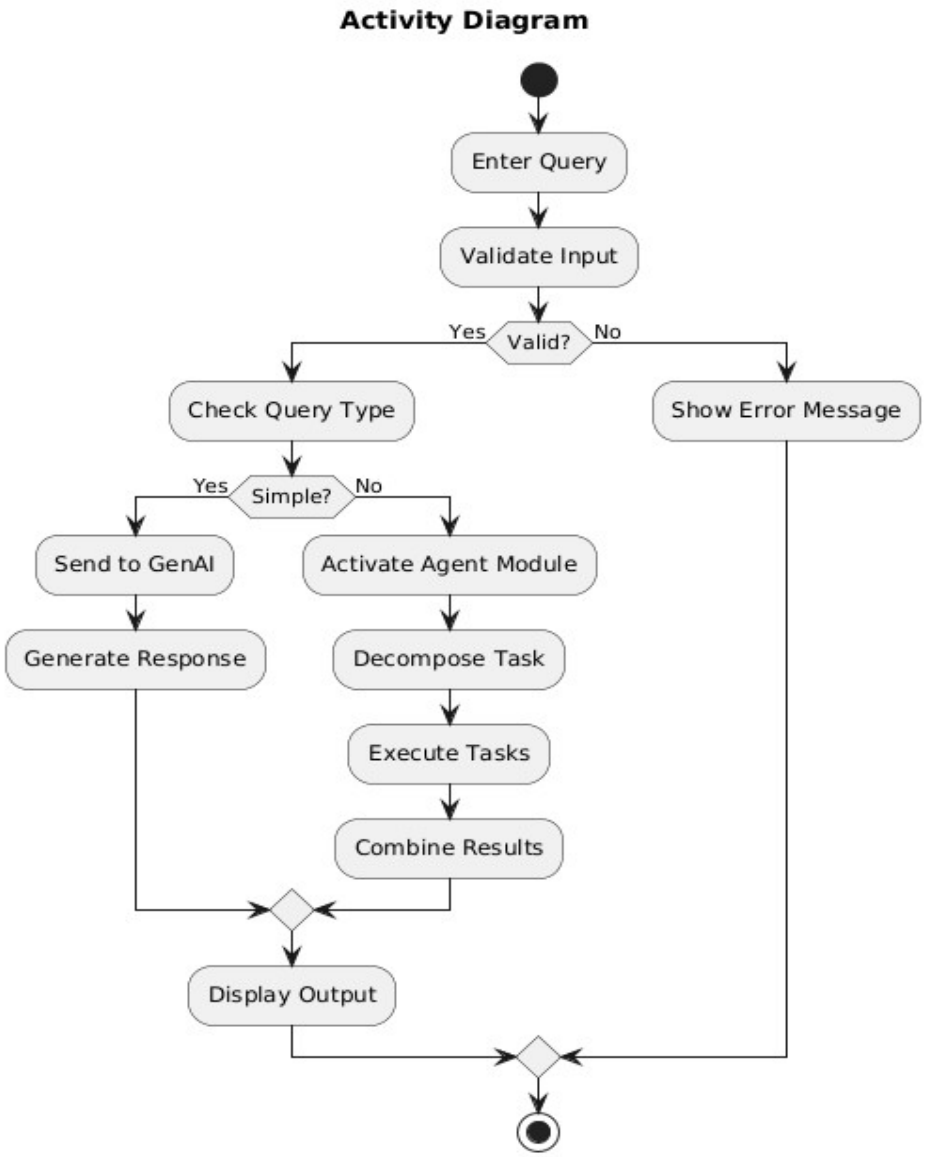

Fig No -3.3
Activity Diagram

3.3.3 Class Diagram

The system includes several classes representing different components. The User class handles user interactions. The Request class represents user input. The Response class stores generated output. The AIProcessor class acts as a central controller that routes requests to either the GenerativeAI class or the AgentModule class.

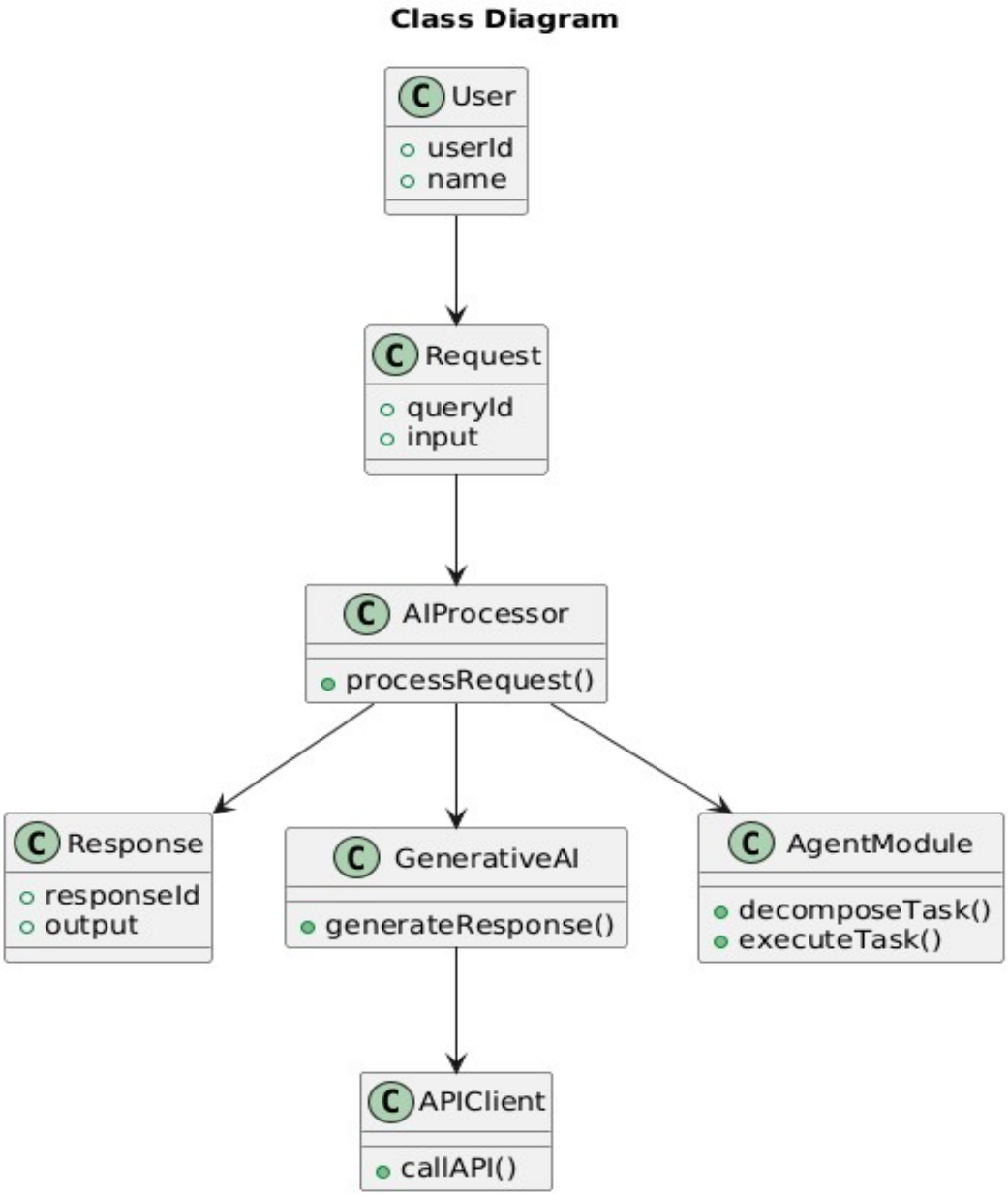

Fig No -3.4
Class Diagram

3.3.4 ER Diagram

The database design includes entities such as User, Query, and Response. The User entity stores user-related information. The Query entity stores user inputs, while the Response entity stores system-generated outputs.

The relationship between User and Query is one-to-many, as a single user can submit multiple queries. Similarly, each query is associated with one response.

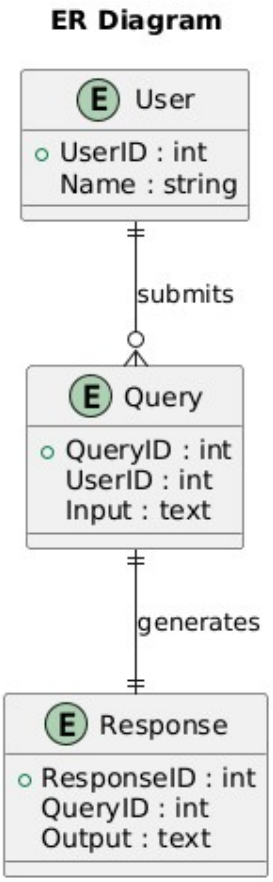

Fig No -3.5
ER Diagram

3.3.5 Use Case Diagram

The primary actor in the system is the user. The user interacts with the system by entering queries and receiving responses. The system performs actions such as validating input, processing queries, generating responses, and displaying results.

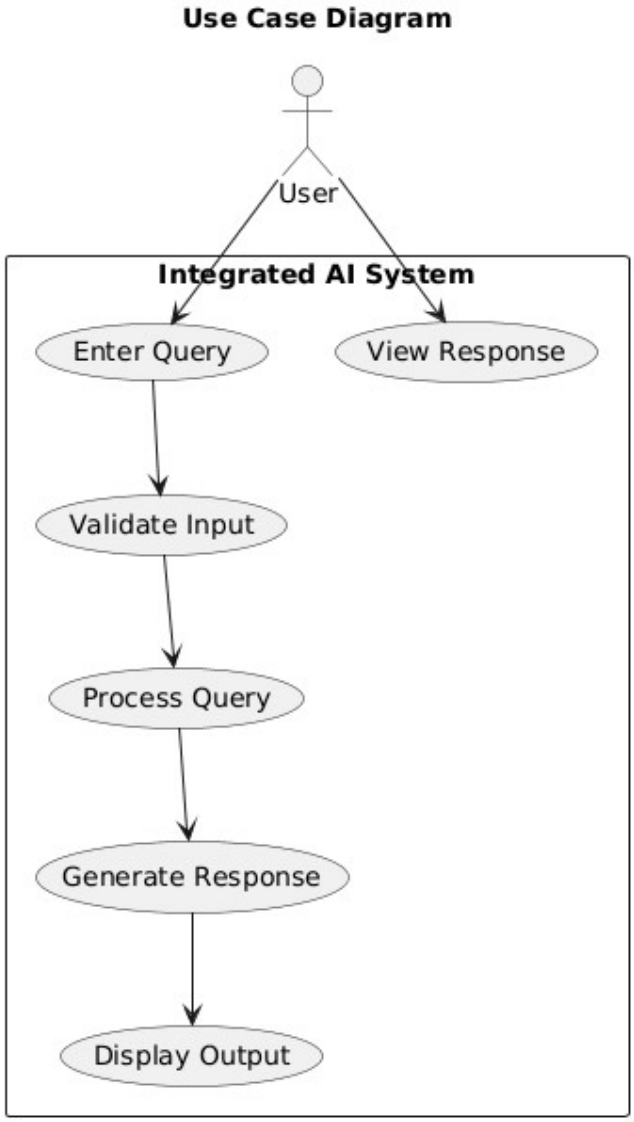

Fig No -3.6
Use Case Diagram

CHAPTER 4

IMPLEMENTATION, TESTING AND MAINTENANCE

4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation

The implementation of the system is based on a full-stack architecture that integrates frontend technologies, backend frameworks, and external AI services. The frontend of the system is developed using HTML, CSS, and JavaScript, which together provide a responsive and interactive user interface. HTML is used to structure the content of the web pages, CSS is used for styling and layout, and JavaScript is responsible for handling user interactions and making asynchronous requests to the backend.

The backend of the system is implemented using FastAPI, a modern Python-based web framework known for its high performance and ease of development. FastAPI is used to define API endpoints, handle HTTP requests, and manage communication between the frontend and the AI modules. Python serves as the core programming language for backend development due to its simplicity and strong support for AI and machine learning libraries.

The system integrates with a Large Language Model API such as Gemini to generate intelligent responses. This API is accessed through HTTP requests, and the responses are processed and formatted before being returned to the user. The agentic component of the system is implemented using modular Python functions that handle task decomposition and execution.

For development purposes, an Integrated Development Environment such as Visual Studio Code is used. It provides features such as syntax highlighting, debugging tools, and extensions for Python and JavaScript development. Version control is managed using Git and GitHub, allowing efficient tracking of changes and collaboration.

Additional tools and technologies include REST APIs for communication, JSON for data exchange, and web browsers for testing and execution. The system is designed to be platform-independent, allowing it to run on different operating systems without major modifications.

4.2 Testing Techniques and Test Plans

Testing plays a crucial role in ensuring the reliability and correctness of the system. The testing process is carried out at multiple levels to verify that each component functions as expected and that the overall system performs efficiently.

Unit testing is performed on individual modules such as input validation, routing logic, and API communication. Each function is tested independently to ensure that it produces the correct output for a given input. This helps in identifying errors at an early stage of development.

Integration testing is conducted to verify the interaction between different modules. The communication between the frontend and backend, as well as the integration with the external AI API, is tested to ensure seamless data flow. Special attention is given to handling API responses and managing errors during communication.

System testing is performed on the complete application to evaluate its overall functionality. Different types of user queries, including simple and complex ones, are tested to ensure that the system correctly routes and processes them. Performance testing is also conducted to evaluate the response time and scalability of the system under different loads.

The test plan includes defining test cases, expected outputs, and evaluation criteria. Test cases cover various scenarios such as valid input, invalid input, API failure, and high load conditions. The results of testing are analyzed to identify issues and improve system performance.

Table No.4.1
Test Cases

Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Status
TC01	Valid simple query processing	"What is AI?"	System returns a meaningful generated answer using GenAI module	As expected	Pass
TC02	Valid complex query processing	"Plan a study schedule for exams"	System uses agent module and returns structured multi-step response	As expected	Pass
TC03	Empty input validation	"" (blank input)	System displays error message asking for valid input	As expected	Pass
TC04	Invalid input (special characters only)	"@@@###"	System rejects input or asks for proper query	As expected	Pass
TC05	API response handling	Valid query	System receives and displays response from LLM API correctly	As expected	Pass
TC06	API failure handling	Valid query during API downtime	System shows error message or fallback response	As expected	Pass
TC07	Frontend to backend communication	Submit query from UI	Backend receives request and processes it correctly	As expected	Pass
TC08	Response formatting	Complex query	Output is properly structured and readable	As expected	Pass
TC09	Multiple requests handling	Multiple queries at once	System handles concurrent requests without crashing	As expected	Pass
TC10	Performance test	Large input query	System responds within acceptable time limit	As expected	Pass
TC11	Routing logic verification	Mixed queries	System correctly routes to GenAI or Agent module	As expected	Pass

Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Status
TC12	Security validation	Script injection input	System sanitizes input and prevents execution	As expected	Pass
TC13	UI interaction test	User clicks submit button	Query is processed and response is displayed	As expected	Pass
TC14	Error message display	Invalid API key	Proper error message is shown to user	As expected	Pass
TC15	Data consistency test	Same query multiple times	System produces consistent and logical responses	As expected	Pass

4.3 Installation Instructions

The installation of the system involves setting up both the backend and frontend components. The backend requires Python to be installed on the system. After installing Python, the required dependencies can be installed using a package manager such as pip. These dependencies include FastAPI, Uvicorn, and any libraries required for API communication.

Once the dependencies are installed, the backend server can be started using a command such as running the FastAPI application through Uvicorn. This launches the server on a local host, making the API endpoints accessible.

The frontend does not require complex installation. It consists of HTML, CSS, and JavaScript files that can be opened directly in a web browser. Alternatively, a local development server can be used to serve the frontend files for better performance.

To integrate the frontend with the backend, the API endpoints must be correctly configured in the JavaScript code. The system should be tested after installation to ensure that the frontend can successfully communicate with the backend and receive responses.

4.4 End User Instructions

The system is designed to be simple and user-friendly, allowing users to interact with it without requiring technical knowledge. To use the system, the user needs to open the application in a web browser. The main interface provides an input field where the user can enter a query.

Once the query is entered, the user can submit it using a button provided on the interface. The system processes the query and displays the response on the screen. If the query is simple, the response is generated quickly using the generative AI module. For complex queries, the system may take slightly longer as it performs multiple steps internally.

CHAPTER 5

RESULTS AND DISCUSSIONS

5.1 User Interface Representation

The user interface of the system is designed to provide a clean and intuitive experience for users interacting with the integrated AI platform. The interface is developed using HTML, CSS, and JavaScript, ensuring that it is lightweight, responsive, and accessible across different devices and browsers.

The main screen consists of an input section where users can enter their queries and a display section where responses are shown. The layout is structured in a way that minimizes complexity and allows users to focus on interaction with the system. The design emphasizes clarity, with proper spacing, readable fonts, and visually distinct sections for input and output.

When a user submits a query, the interface dynamically updates to display the response without requiring a page reload. This is achieved through asynchronous communication with the backend. The interface also handles loading states, giving users feedback that their request is being processed.

Error messages are displayed in a clear and informative manner whenever invalid input is provided or when system errors occur. Overall, the interface ensures smooth interaction between the user and the AI system.

5.2 Brief Description of Various Modules of the System

The system is composed of several interconnected modules, each responsible for a specific function. The frontend module handles user interaction and communicates with the backend through API calls. It is responsible for capturing user input and displaying responses.

The backend module acts as the core processing unit. It manages request handling, input validation, routing logic, and communication with external AI services. This module ensures that requests are processed efficiently and that responses are generated correctly.

The generative AI module is responsible for handling straightforward queries. It communicates with the Large Language Model API to generate human-like responses. This module is optimized for speed and accuracy in handling direct questions.

The agentic AI module is designed for complex queries that require multiple steps or logical reasoning. It decomposes tasks into smaller components, executes them, and combines the results to produce a coherent output.

The API integration module manages communication with external services such as the Gemini API. It ensures secure and efficient data exchange between the system and the AI models.

Each module operates independently but is integrated seamlessly to provide a unified system.

5.3 Snapshots of System with Brief Detail of Each

The developed AgriTriage platform consists of multiple interactive modules designed to support agricultural emergency response and educational assistance. The following screenshots represent the major functionalities implemented in the system.

5.3.1 AgriTriage Triage Analysis Module

This screen represents the main triage interface of the AgriTriage platform. The user enters sender information including name, email address, and the farmer's message related to agricultural issues. After submitting the message, the agentic AI pipeline processes the input and generates a structured analysis including urgency level, urgency score, identified intent, extracted entities, and a professional draft response. In the illustrated example, the system identifies a crop disease issue related to a lemon tree and generates a medium urgency classification along with treatment recommendations.

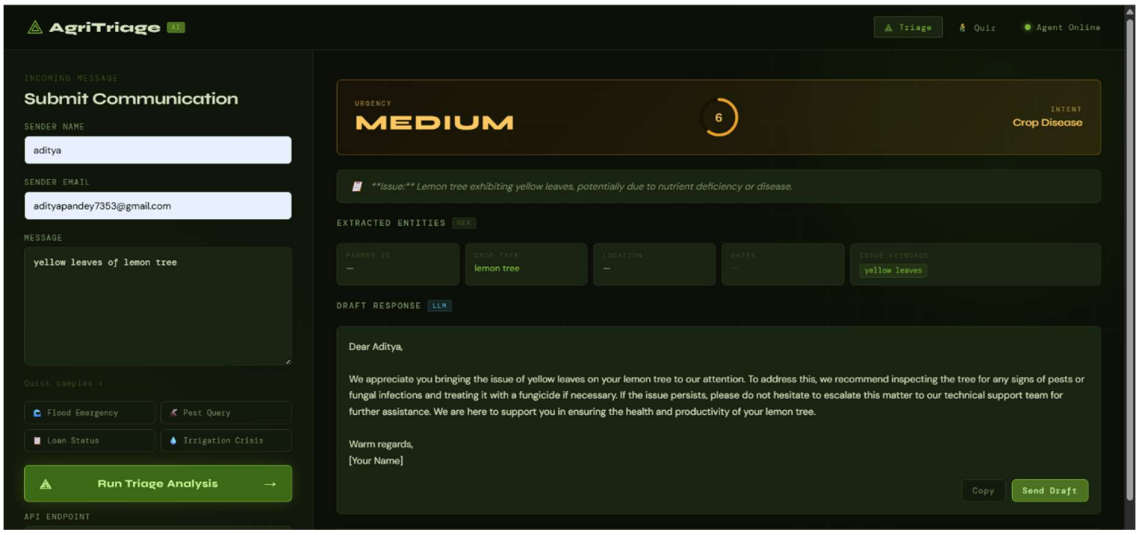

Figure 5.1

5.3.2 FarmWise Quiz Module

This screen represents the FarmWise educational quiz module integrated within the platform. Users can select quiz topics and answer agriculture-related multiple-choice questions. After selecting an answer, the system evaluates user performance and provides feedback. This module promotes agricultural learning and knowledge retention among farmers and agricultural workers through interactive AI-supported assessments.

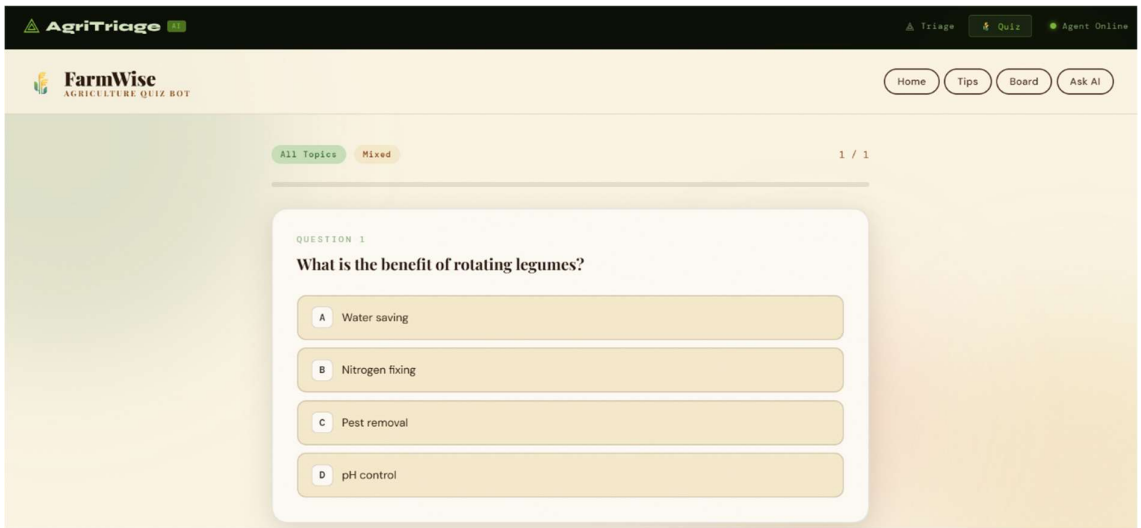

Figure 5.2

Chapter 6

Summary and Conclusions

The AgriTriage project presents the design and development of an intelligent agricultural support platform that combines the capabilities of Generative AI and Agentic AI within a unified system. The primary objective of the project was to create a practical solution that assists farmers, agricultural support officers, and learners by providing automated emergency triage, intelligent information extraction, educational quiz-based learning, and AI-powered agricultural guidance. The developed system successfully demonstrates how modern artificial intelligence technologies can be applied to solve real-world challenges in the agricultural domain.

The system was implemented using a full-stack architecture consisting of a frontend developed with HTML, CSS, and JavaScript, and a backend developed using Python and FastAPI. The agentic processing pipeline was implemented using LangChain, while Groq's inference infrastructure and Meta's Llama language models were used for fast and accurate language processing. Supabase was integrated as the cloud database solution for storing quiz sessions and leaderboard data. This combination of technologies enabled the system to deliver scalable, responsive, and intelligent services.

During the implementation phase, multiple modules were developed and integrated successfully. The triage module was designed to analyze farmer distress messages, classify urgency levels, identify user intent, extract important entities, and generate professional response drafts. The FarmWise educational module was designed to improve agricultural awareness through interactive quizzes, performance tracking, and AI-generated explanations. Additional features such as leaderboard management and AI-based question answering further enhanced the usability and educational value of the platform.

Comprehensive testing was conducted on different system modules to verify functionality, performance, reliability, and error handling. The test results demonstrated that the system performs effectively under normal operating conditions and produces accurate, structured, and meaningful outputs. The use of agentic workflows improved task handling for complex queries, while generative AI ensured natural and context-aware communication.

In conclusion, the AgriTriage project successfully achieved its intended objectives by developing an intelligent and scalable agricultural support system. The project demonstrates the practical potential of combining Generative AI and Agentic AI in real-world applications. The developed platform provides a strong foundation for future improvements and wider deployment in agricultural advisory systems, farmer education platforms, and rural technology initiatives.

CHAPTER 7

FUTURE SCOPE

The developed system demonstrates the effective integration of Generative AI and Agentic AI; however, there are several opportunities for further enhancement and expansion. Future improvements can focus on increasing the intelligence, scalability, and real-world applicability of the system.

One of the primary areas for future work is the enhancement of the agentic module. Currently, the system performs task decomposition and execution at a basic level. This can be extended by incorporating advanced planning algorithms, memory mechanisms, and context retention so that the system can handle long-term interactions and more complex workflows. Introducing frameworks such as multi-agent collaboration can further improve problem-solving capabilities.

Another important direction is the integration of a persistent database and user management system. By storing user interactions, preferences, and history, the system can provide personalized responses and improve over time. This would also enable features such as user authentication, session tracking, and analytics for better system monitoring.

The system can also be expanded to support multimodal capabilities. Instead of being limited to text-based input, future versions can accept images, audio, or documents as input and generate corresponding outputs. This would significantly increase the usability of the system in domains such as healthcare, education, and agriculture.

Performance optimization is another area of improvement. Techniques such as caching, load balancing, and asynchronous task execution can be implemented to reduce response time and improve system scalability. Deployment on cloud platforms can further enhance availability and support a larger number of users.

Additionally, the system can be extended to domain-specific applications. For example, it can be customized for use in customer support, education platforms, business automation, or smart assistants. Integrating domain-specific datasets and fine-tuned models will improve accuracy and relevance in these areas.

Security and privacy can also be strengthened in future versions. Implementing secure API handling, data encryption, and access control mechanisms will ensure safe handling of user data. Compliance with data protection standards can further improve trust and reliability.

In summary, the system provides a strong foundation that can be extended in multiple directions. With advancements in AI technologies and further development, the system has the potential to evolve into a highly intelligent and widely applicable solution across various domains.

Appendix

Appendix A: Sample Source Code

The following code snippet represents the backend API implementation for processing farmer messages in the AgriTriage system.

Python Run

```python
@app.post("/triage")
async def triage_message(request: TriageRequest):
    result = await process_farmer_query(request.message)
    return result
```

The following code snippet represents the quiz leaderboard storage logic using Supabase.

Python Run

```python
supabase.table("quiz_sessions").insert({
    "username": username,
    "score": score,
    "topic": topic
}).execute()
```

Appendix B: Sample Input and Output

Sample Input:

Farmer Message: My tomato crop leaves are turning yellow and insects are spreading rapidly in the field.

Sample Output:


```
{
 "urgency": "HIGH",
 "score": 9,
 "crop": "Tomato",
 "issue": "Pest Attack",
 "response": "Immediate pesticide inspection is recommended."
}
```

Appendix C: Database Schema

Quiz Sessions Table:

Field Name	Data Type	Description
id	Integer	Primary Key
username	Varchar	User Name
score	Integer	Quiz Score
topic	Varchar	Quiz Topic
created_at	Timestamp	Session Date

Appendix D: Test Case Details

Test Case ID	Description	Result
TC01	Triage API Response	Pass
TC02	Quiz Generation	Pass
TC03	Leaderboard Storage	Pass
TC04	Invalid Input Handling	Pass

Appendix E: Installation Commands

```
# Install dependencies
uv sync
# Run backend
uv run uvicorn app.main:app --reload
# Open frontend
Open index.html in browser
```

Appendix F: User Guide

To use the system, the user first opens the AgriTriage web application in a supported web browser such as Google Chrome, Mozilla Firefox, or Microsoft Edge. After the application loads successfully, the main dashboard is displayed, providing access to the two core modules of the platform: the Triage module and the FarmWise educational module.

If the user wants to report an agricultural issue or emergency, they can navigate to the Triage module. In this section, the user enters details such as sender name, email address, and the farming-related message describing the issue. The message may include information related to crop disease, pest attacks, irrigation problems, fertilizer concerns, or other agricultural emergencies. After entering the required details, the user clicks the analysis button to submit the request. The system then processes the input using the agentic AI pipeline and generates structured results including urgency classification, urgency score, detected intent, extracted entities, and a professionally generated draft response.

If the user wants to improve agricultural knowledge, they can switch to the FarmWise module. In this section, users can select quiz topics and answer agriculture-related multiple-choice questions. After each submission, the system evaluates the selected answers, calculates the score, and provides AI-generated explanations to help users understand the correct concepts. Quiz performance data can also be stored for leaderboard ranking and future progress tracking.

The system also provides additional AI-based assistance features such as asking farming-related questions directly to the AI model. Users can enter natural language queries related to crops, soil health, fertilizers, pest control, irrigation, or modern farming techniques and receive informative responses generated by the language model.

If invalid input is entered or any system error occurs, the application displays clear validation messages to guide the user toward proper usage. The interface is designed to be simple, responsive, and easy to use for both technical and non-technical users, ensuring smooth interaction throughout the system.

Reference

[1] Groq Inc. Groq API Documentation. Source: https://console.groq.com/docs/overview

[2] LangChain. LangChain Official Documentation. Source: https://docs.langchain.com

[3] LangChain. LangChain Groq Integration Documentation. Source: https://docs.langchain.com/oss/python/integrations/chat/groq

[4] LangChain. langchain-groq Python Reference. Source: https://reference.langchain.com/python/langchain-groq

[5] Supabase Inc. Supabase Official Documentation. Source: https://supabase.com/docs

[6] FastAPI. FastAPI Official Documentation. Source: https://fastapi.tiangolo.com

[7] Uvicorn Contributors. Uvicorn Documentation. Source: https://www.uvicorn.org

[8] Meta AI. Llama Large Language Model Documentation. Source: https://ai.meta.com/llama

[9] Python Software Foundation. Python Documentation. Source: https://docs.python.org/3

[10] MDN Web Docs. HTML, CSS and JavaScript Documentation. Source: https://developer.mozilla.org